

Subprograms: Overloading

Programming Languages

Millersville University



Function Overloading

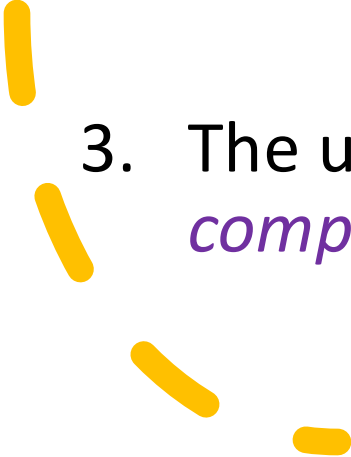
- A function is said to be *overloaded* when the **name** is the same but the **signature** is different
- Usually resolved via argument type analysis

```
int  
median (int x, int y, int z);
```

```
double  
median (double x, double y, double z);
```



Function Overloading Rules

1. The same *function name* is used for more than one function definition
 2. The functions must differ either by *arity* or *types* of their parameters (not return value)
 3. The usage within a program must be resolved at *compile / analysis* time rather than at runtime
- 

Function Overloading

```
// circle
double area (double r) {
    return 3.1415926535 * r * r;
}

// rectangle
double area (double x, double y) {
    return x * y;
}

// triangle
double area (double a, double b, double c) {
    double s = (a + b + c) / 2.0;
    return sqrt (s * (s - a) * (s - b) * (s - c));
}
```

Function Overloading: C++/Java

- Constructors can also be overloaded!

```
class MyAdt {  
    MyAdt ();           // default  
    MyAdt (int size);   // size constructor  
    MyAdt (MyAdt other); // Java - copy constructor  
    MyAdt (const MyAdt& other); // C++ - copy constructor  
    MyAdt (MyAdt&& other); // C++ - move constructor  
    . . .  
}
```



Function Overloading Issues

Pros

Cons



Function Overloading Issues

Pros

- **Versatile** – use names that make sense and are common
- **Customizable** – introduce same behavior for new classes or datatypes

Cons

- Difficult to discern at function call
- Type coercion could result in unintended function being called
- Lots of overloads -> longer compilation / resolution

Attribution

Slides originally developed by William Killian,
modified by Stephanie Schwartz
modified by Gary Zoppetti

Sources include Robert Sebesta's [Concepts of Programming Languages](#)